DM548: Computer Architecture & System Programming
DM588: Computer Architecture

Fall 2026

Cache Memory

- The Memory Hierarchy and the Principle of Locality
- Memory Characteristics
- Memory Hierarchy
- Cache Memory Principles
- Cache Design

  Addresses and Mapping Functions
  Replacement Algorithms
  Write Policy
  Cache Configuration and Hit Ratios

Where are we?

Recall:

- Architecture: what can it do, as seen by the user.
  E.g., instruction set.
- Organisation: the implementation of an architecture.
  E.g., hardware units, interconnections, memory technology.

Today (and next lecture): an overview of

- the memory hierarchy and the principle of locality,
- memory organisation,
- memory technologies (briefly, more later in the course),
- cache design and function.

Organisation

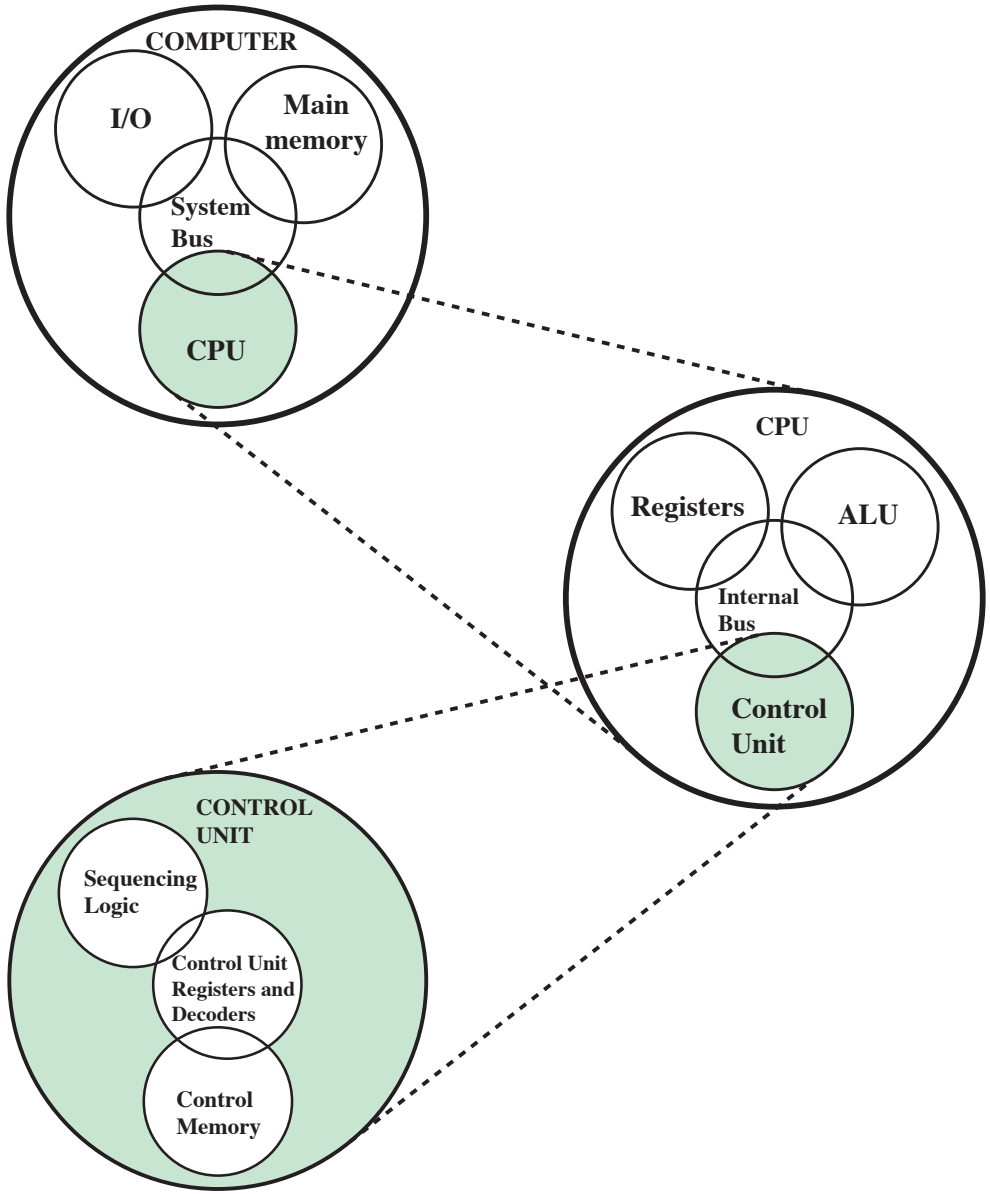


Figure 1.1 A Top-Down View of a Computer

- The Memory Hierarchy and the Principle of Locality
- Memory Characteristics
- Memory Hierarchy
- Cache Memory Principles
- Cache Design

  Addresses and Mapping Functions
  Replacement Algorithms
  Write Policy
  Cache Configuration and Hit Ratios

Principle of Locality

Also known as locality of reference.

Consider the execution of a program:

- Instructions are generally fetched sequentially (except for branches and calls).
- Generally execution is confined to few procedures at a time.
- Most loop constructs consist of a relatively small number of instructions, and are repeated many times.
- In most programs, most of the computation involves arrays of some sort. Often, successive references are made to elements located closely together.

Therefore:

- memory locations are accessed non-uniformly,
- the distribution of locations accessed changes slowly over time,
- the correlation between immediate past and future locations is very high.

Principle of Locality

Two types of locality:

- Temporal locality: memory accessed in the recent past tend to be access in the near future. E.g., loops or stack data.
- Spatial locality: when some location is referenced, locations in a limited window around it tend to be accessed next. E.g., instructions or arrays of data.

Cache memory:

- Exploits temporal locality by keeping recently referenced data close to the CPU, and having a hierarchy of caches.
- Exploits spatial locality by keeping blocks of data instead of singular bytes/words.

Note: Now that we know the hardware exploits it, we as programmers can rearrange our algorithms to increase locality of reference. Performance may increase dramatically.

Locality bimodal: instructinos and data

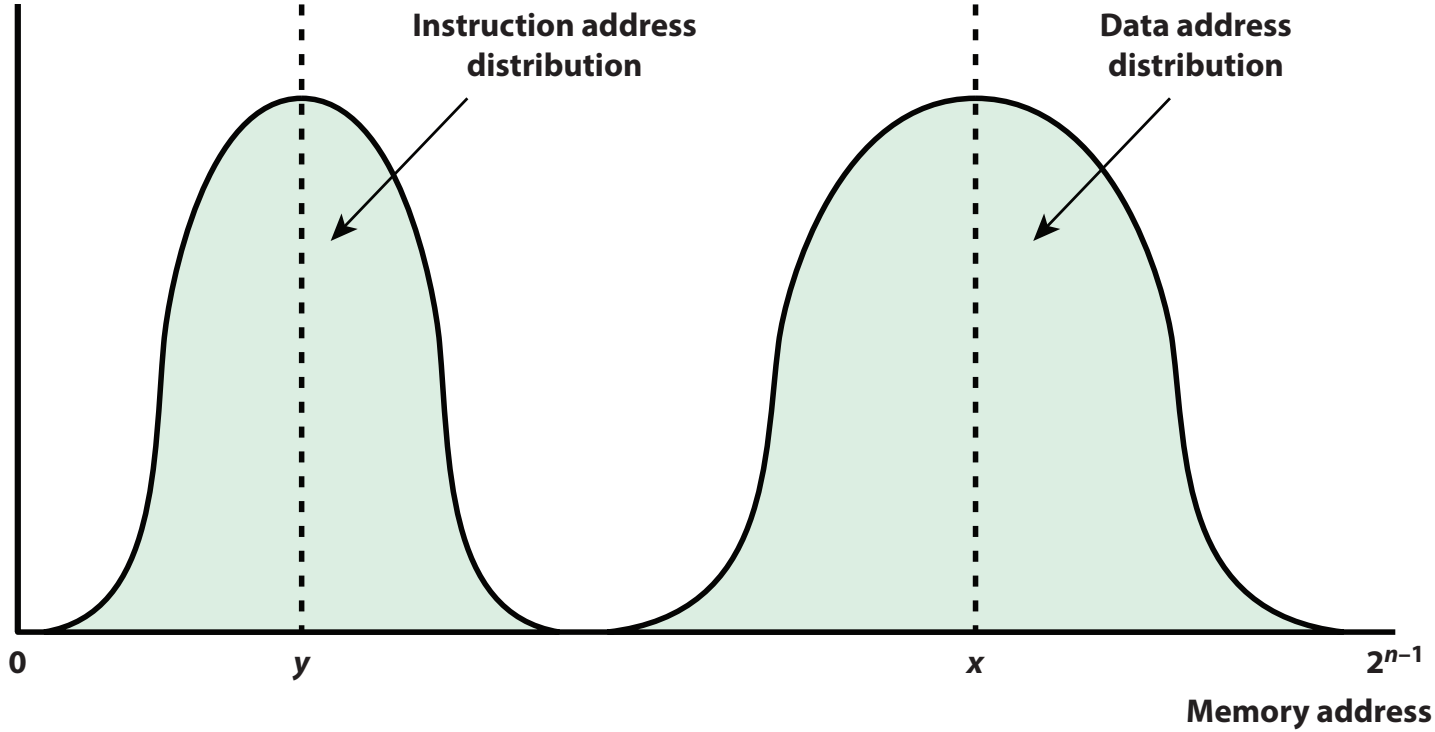


Figure 4.3 Idealized Spatial Locality Behavior: Probability Distribution for Next Memory Access (most recent data memory access at location *x*; most recent instruction fetch at location *y*)

At a given time, the distribution of data accesses tend to have more variation than the distribution for instructions.

- The Memory Hierarchy and the Principle of Locality
- Memory Characteristics
- Memory Hierarchy
- Cache Memory Principles
- Cache Design

  Addresses and Mapping Functions
  Replacement Algorithms
  Write Policy
  Cache Configuration and Hit Ratios

Memory System Characteristics

- Location
  - internal
  - external
- Capacity, expressed as
  - number of bytes, or
  - number of words
- Unit of transfer
  - byte
  - word
  - larger blocks
- Access method
  - sequential
  - direct
  - random
  - associative
- Performance
  - access time
  - cycle time
  - transfer rate
- Physical type
  - semiconductor
  - magnetic
  - optical
  - magneto-optical
- Physical characteristics
  - volatile/non-volatile
  - erasable/non-erasable
- Organisation
  - memory modules

Memory System Characteristics

- Location
  - Relative to how the CPU can access it.
  - Internal is, e.g., CPU registers, caches, main memory.
  - External are the peripheral devices,
    i.e., those attached via an IO module.
    E.g., optical disks, magnetic disks, SSDs, tapes, USB sticks.

Memory System Characteristics

- Location
  - Relative to how the CPU can access it.
  - Internal is, e.g., CPU registers, caches, main memory.
  - External are the peripheral devices,
    i.e., those attached via an IO module.
    E.g., optical disks, magnetic disks, SSDs, tapes, USB sticks.
- What is a "word" of memory? Its complicated . . .
  - The "natural" unit that the CPU works in, e.g., width of registers.
  - Often related to the number of bits used for integers.
  - Counter example: CRAY C90 (launched in 1991);
    integer width was 48 bits, but word size was 64.
  - x86: many different integer widths, but word size is usually denoted as 32 bits.
  - x86-64: as x86 but word size 64 bits.
  - For internal memory the word size is often relevant (see later).
- How about a "byte"?
  - Historically a byte could have varying size.
  - The de facto standard is now: 1 byte = 8 bits.

Memory System Characteristics

- Unit of transfer
  - How much memory is being transferred at a time?
  - E.g., for a slow external disk we may as well read a large block of data instead of individual bytes.
  - From the lowest cache level to the CPU it may be a byte or word.
  - From main memory to the last cache level it is a small block, e.g., 64 bytes.
- Addressable unit
  - How much data is located at each address?
  - In x86/x86-64 we have seen that each address contains 1 byte.
  - In some systems the smallest unit is 1 word.
  - In basically all modern machines it is 1 byte.
  - With A address bits we can refer to $N = 2^A$ locations.
  - When we go into the details there are really multiple answers:
    - The main memory and caches deals with words.
    - The CPU can then pick specific bytes out of a word.

Access Method

- Sequential access
  - Example: magnetic tapes.
  - An addressable unit is called a record.
  - We can only go to the next or previous record.
  - There is a shared read-write mechanism.
  - Access time: highly variable.
- Direct access
  - Example: hard disk drives.
  - Also using a shared read-write mechanism.
  - The address of a block/record corresponds to a unique physical location.
  - We can jump to the general vicinity of a block, but then need a bit of sequential search.
  - Access time: variable.

Access Method

- Random access
  - Example: main memory (hence "RAM"), some caches
  - The addressing mechanism can select every location directly.
  - Access time: constant, independent of location.
- Associative access
  - Examples: associative and set-associative caches.
  - Similar to random access.
  - But, the specific location is based on (part) of its content.
  - Access time: constant, independent of location and content.

Performance Characteristics

- Access time (latency)
  - Random-access memory: time for a read/write operation. That is, time from the memory gets the address until the data is retrieved/stored.
  - Other access patterns: time to position the read/write mechanism.
- Memory cycle time (primarily for RAM)
  - Access time + delay until the next address can be given to the memory.
  - Signals may need to die out, or the data need to be regenerated if reads are destructive.
- Transfer rate
  - RAM: $\frac{1\,\text{unit}}{\text{cycle time}}$
  - Other access patterns: $T_n = T_A + \frac{n}{R}$, with
    T_n: average time to read n bits
    T_A: average access time
    n: number of bits
    R: transfer rate, in bits per second

- The Memory Hierarchy and the Principle of Locality
- Memory Characteristics
- Memory Hierarchy
- Cache Memory Principles
- Cache Design

  Addresses and Mapping Functions
  Replacement Algorithms
  Write Policy
  Cache Configuration and Hit Ratios

The Memory Dilemma

- How much?
- How fast?
- How expensive?

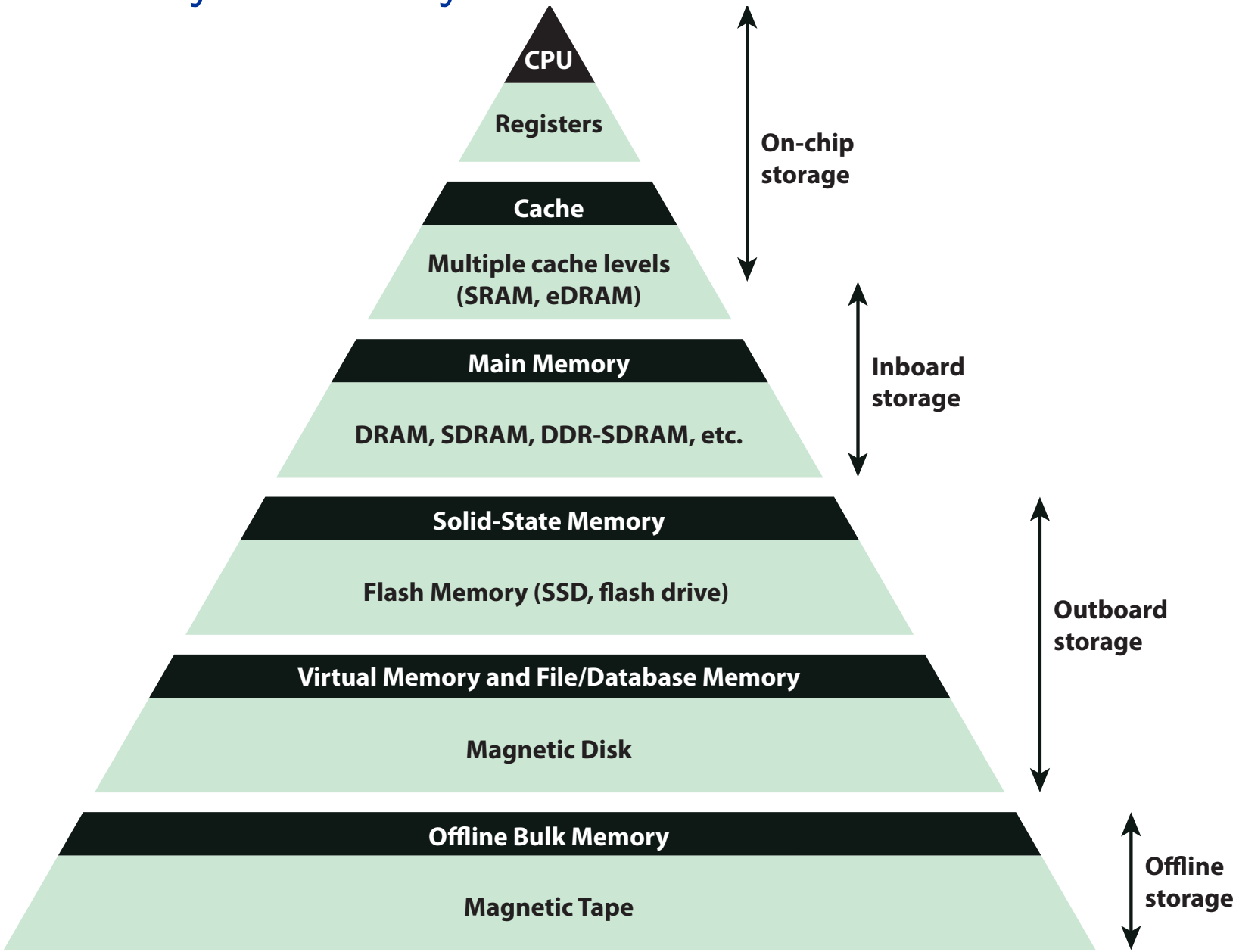
Generally, the trade-offs are:

- Faster access time $\Rightarrow$ higher cost per bit.
- Greater capacity $\Rightarrow$ lower cost per bit.
- Greater capacity $\Rightarrow$ slower access.

"Solution":

- Use multiple technologies in multiple components.
- Arrange it all in a hierarchy for access.

The Memory Hierarchy



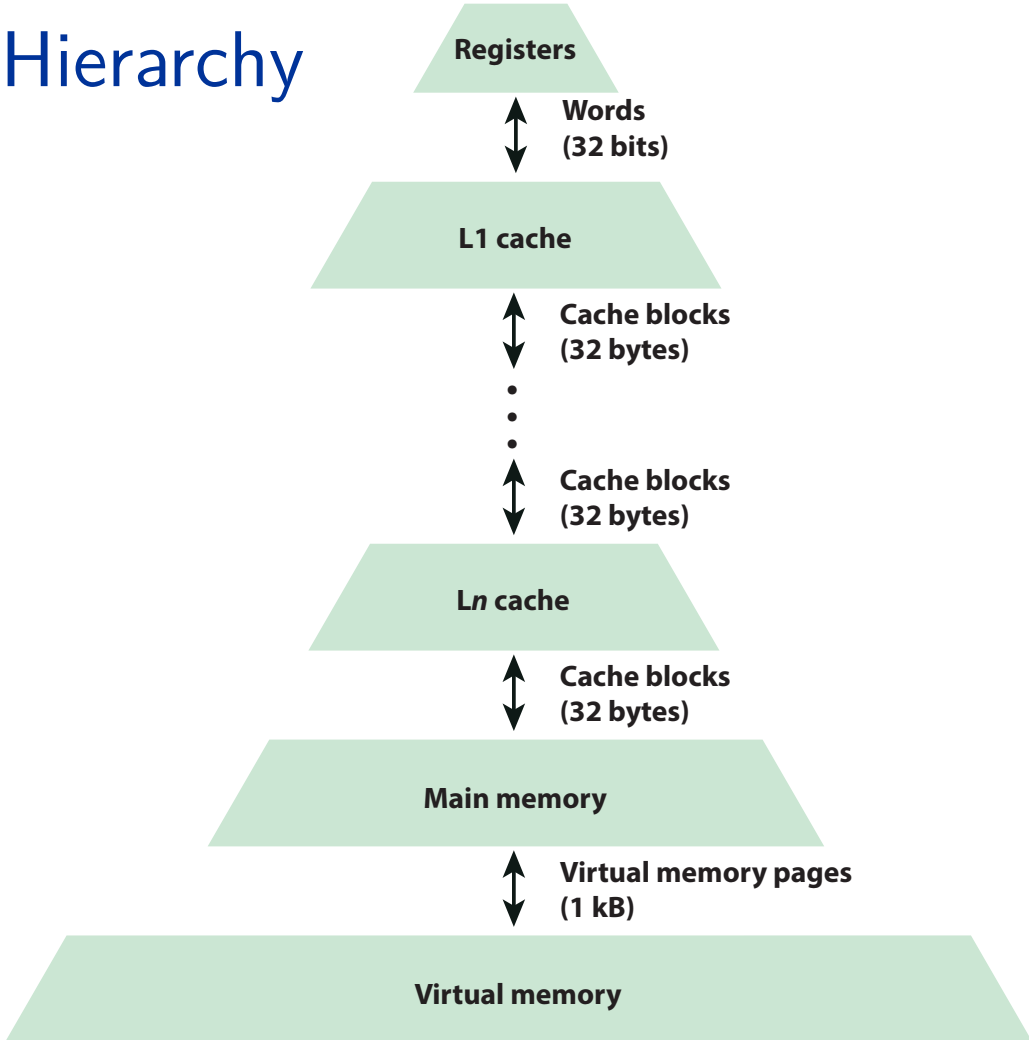
The Memory Hierarchy

Going from top to bottom:

- Decreasing cost per bit.
- Increasing capacity.
- Increasing access time.
- Decreasing frequency of access by the CPU.

The last part is the is the key to success: locality of reference.

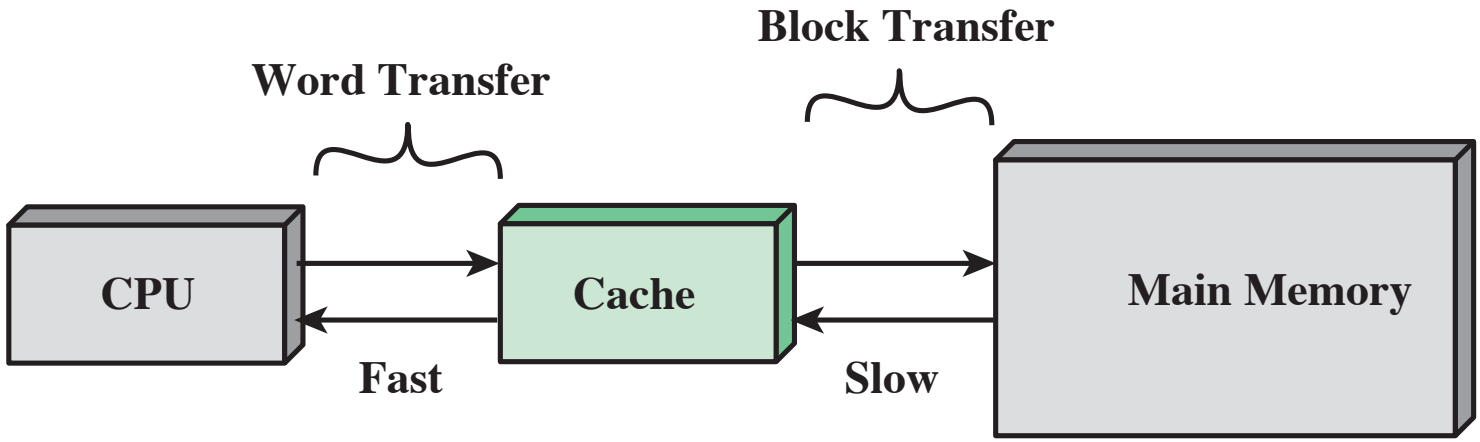
The Memory Hierarchy



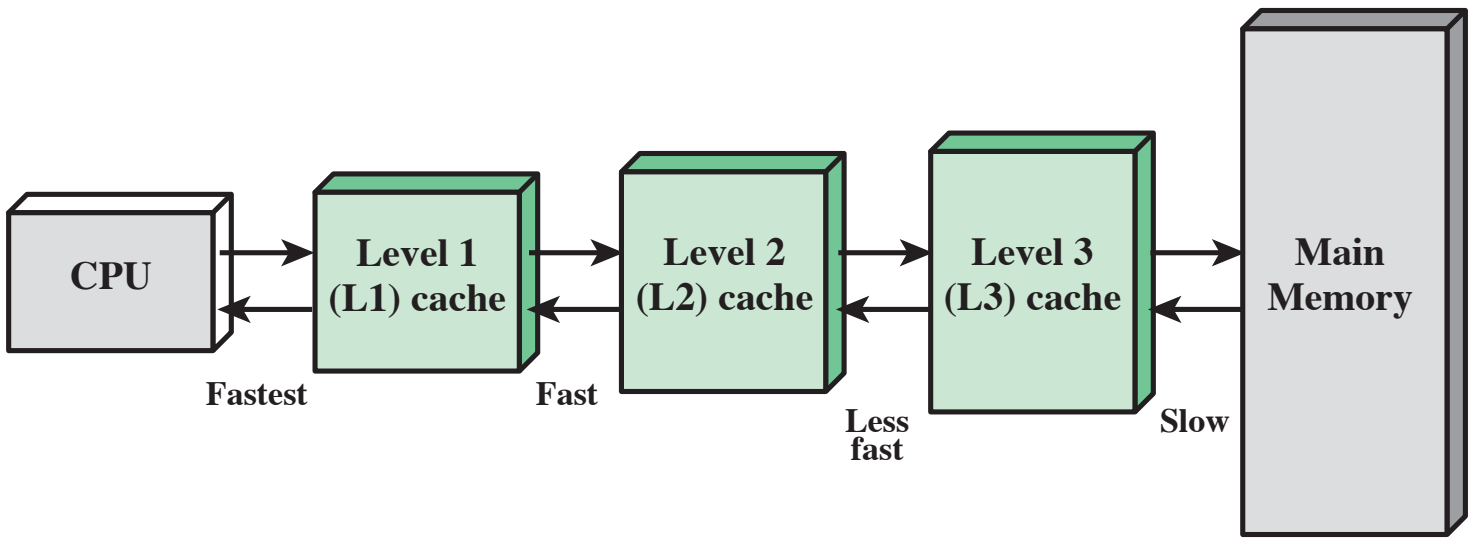
- In modern x86-64 systems the cache blocks are 64 bytes.
- Some cache levels are duplicated for each CPU core, some are shared.
- Virtual memory is a topic in DM590: Operating Systems.

- The Memory Hierarchy and the Principle of Locality
- Memory Characteristics
- Memory Hierarchy
- Cache Memory Principles
- Cache Design
  - Addresses and Mapping Functions
  - Replacement Algorithms
  - Write Policy
  - Cache Configuration and Hit Ratios

A Typical Example



(a) Single cache



(b) Three-level cache organization

Cache and Main Memory Structure

As seen from a byte-addressable CPU:

- Each address specifies a byte.
- Bytes are grouped into words.
  There is always some power of 2 bytes per word (e.g., 2, 4, or 8).
- Words are grouped into blocks.
  There is always some power of 2 words per block.

Cache and Main Memory Structure

As seen from a byte-addressable CPU:

- Each address specifies a byte.
- Bytes are grouped into words.
  There is always some power of 2 bytes per word (e.g., 2, 4, or 8).
- Words are grouped into blocks.
  There is always some power of 2 words per block.

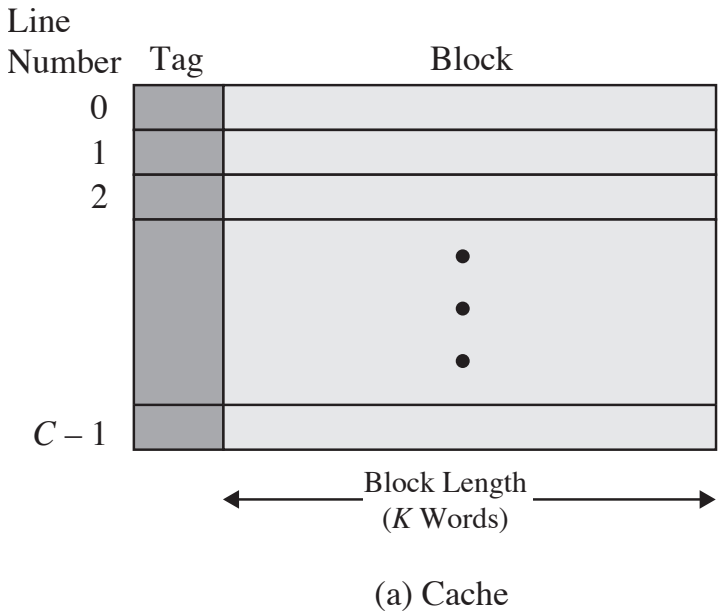
Unfortunately the book is a bit confusing in the next part.

- In the text they often write "words or bytes",
  but in the figures only "words",
  and in concrete examples they usually mean "bytes".
- A word can vary in size depending on machine.
- (Not to be confused with the "word" in x86 assembly)

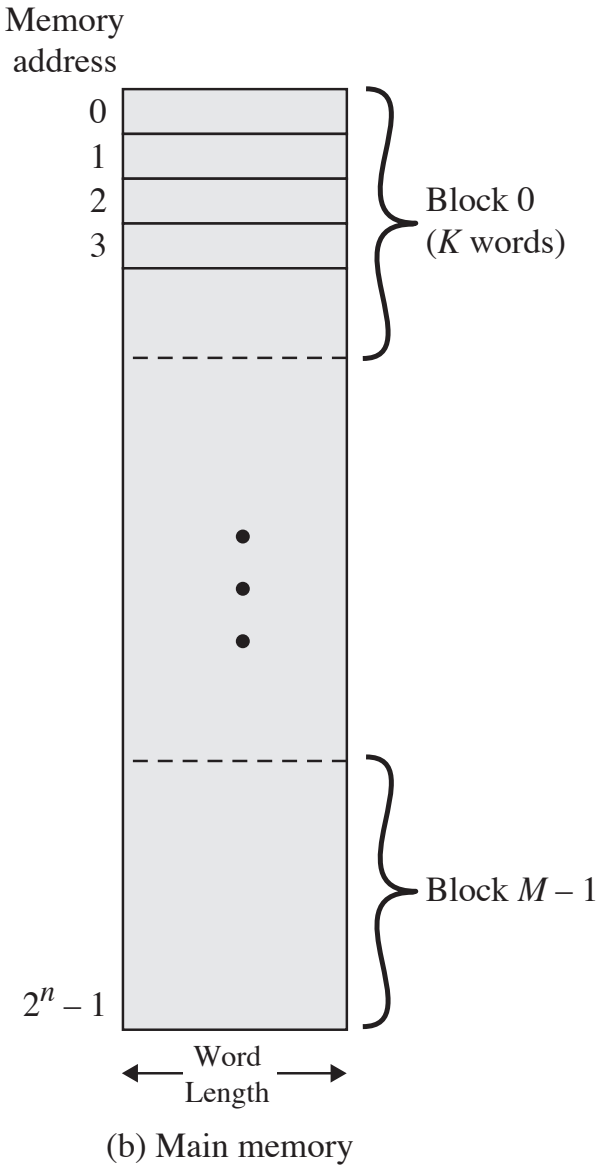
Simplifying assumption for the next figures:

1 word = 1 byte

Cache and Main Memory Structure



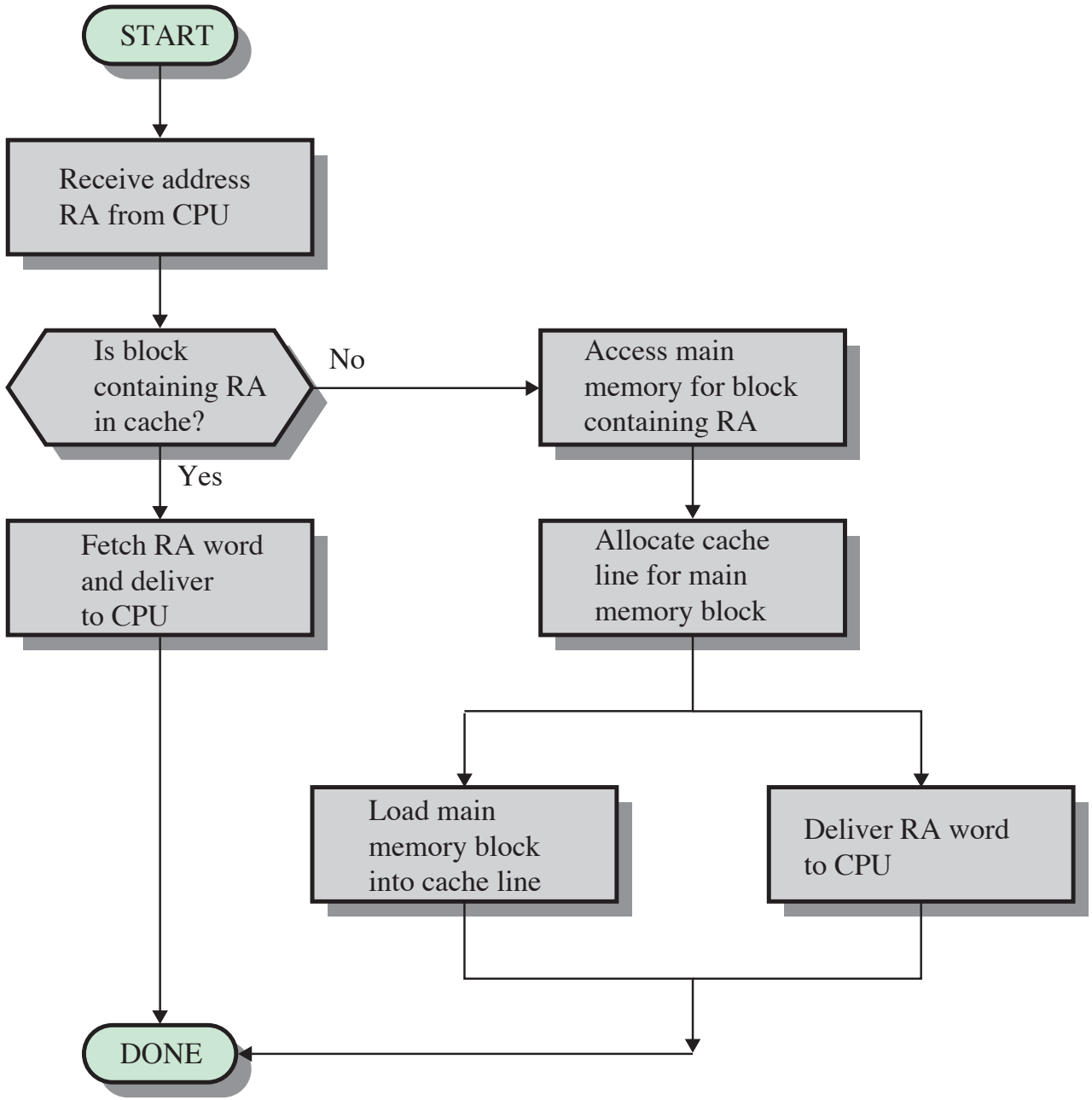
(a) Cache



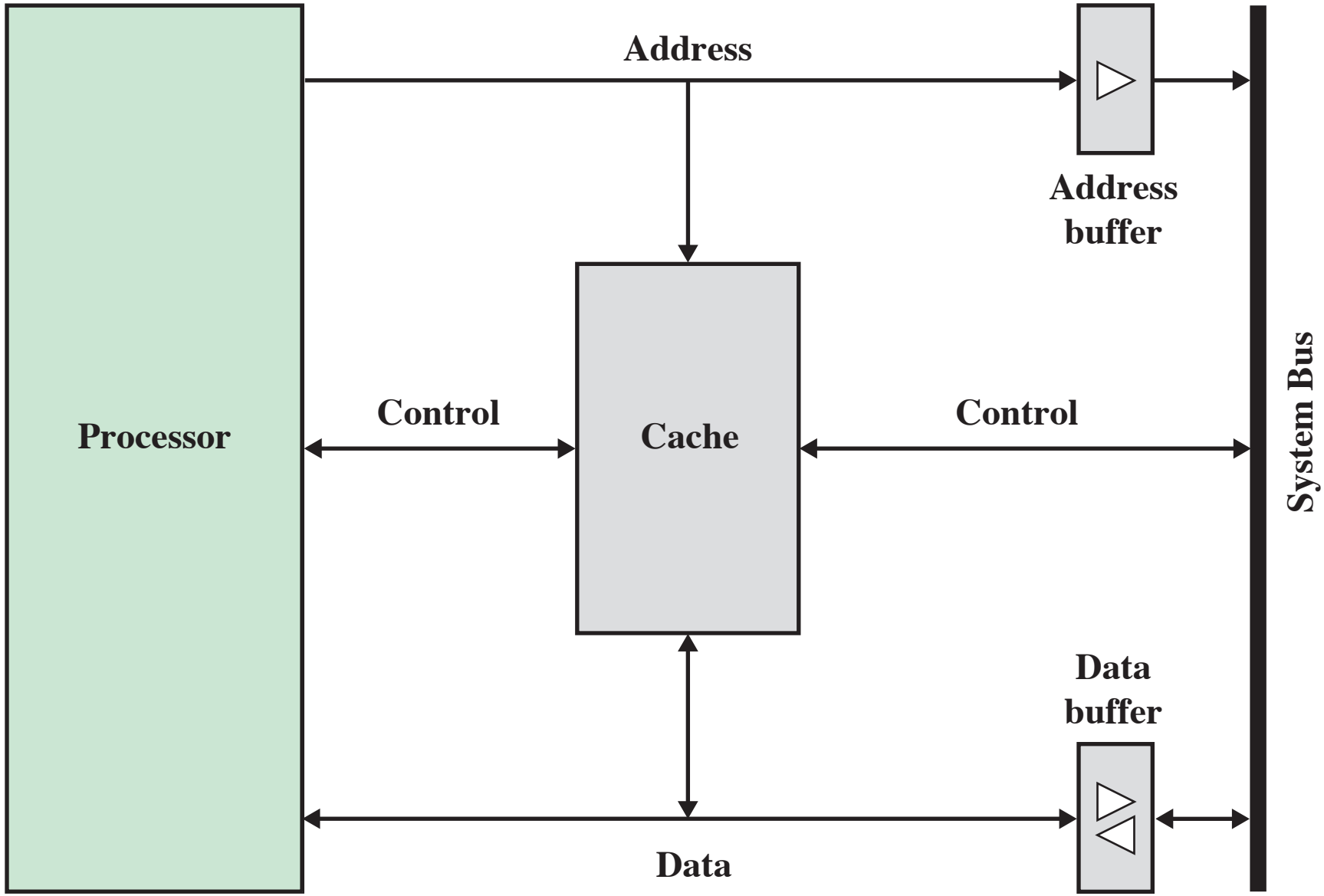
(b) Main memory

The number of cache lines is considerably smaller than the number of blocks in main memory.

Cache Read Flowchart



Typical Cache Organization



- The Memory Hierarchy and the Principle of Locality
- Memory Characteristics
- Memory Hierarchy
- Cache Memory Principles
- Cache Design

  Addresses and Mapping Functions
  Replacement Algorithms
  Write Policy
  Cache Configuration and Hit Ratios

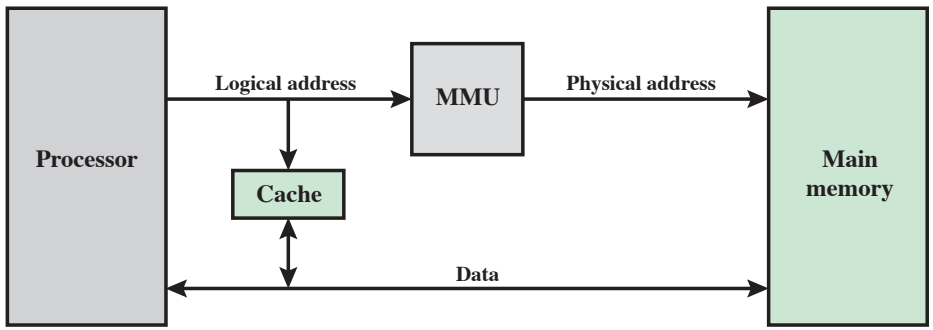
Elements of Cache Design

- Cache addresses
  - logical
  - physical
- Cache size
- Mapping function
  - direct
  - associative
  - set-associative
- Replacement algorithm
  - Least recently used (LRU)
  - First-in-first-out (FIFO)
  - Least frequently used (LFU)
  - Random
  - ...
- Write policy
  - Write through
  - Write back
- Line size
- Number of caches
  - Number of levels
  - Unified or split

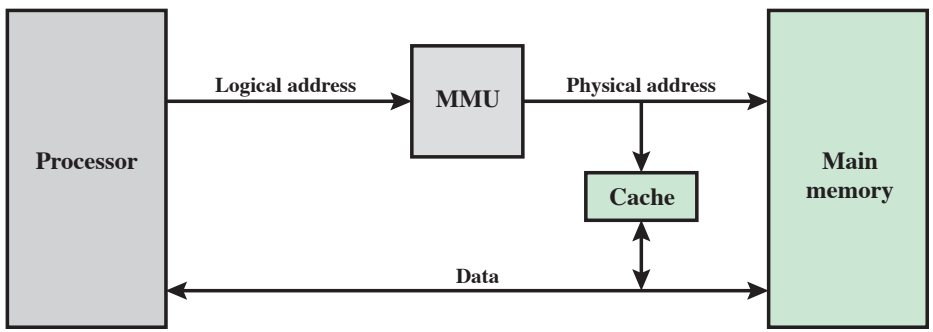
- The Memory Hierarchy and the Principle of Locality
- Memory Characteristics
- Memory Hierarchy
- Cache Memory Principles
- Cache Design
  - Addresses and Mapping Functions
  - Replacement Algorithms
  - Write Policy
  - Cache Configuration and Hit Ratios

Cache Addresses

- The addresses we see in assembly are not physical.
- They are logical/virtual addresses the OS and hardware has set up.
- This is in part due to security considerations.
- When we read/write each logical address is translated to the real physical address.
- This is done by the memory management unit (MMU).
- Should we cache based on logical or physical addresses?
- A very complicated topic.
- We will just look at the cache.



(a) Logical Cache



(b) Physical Cache

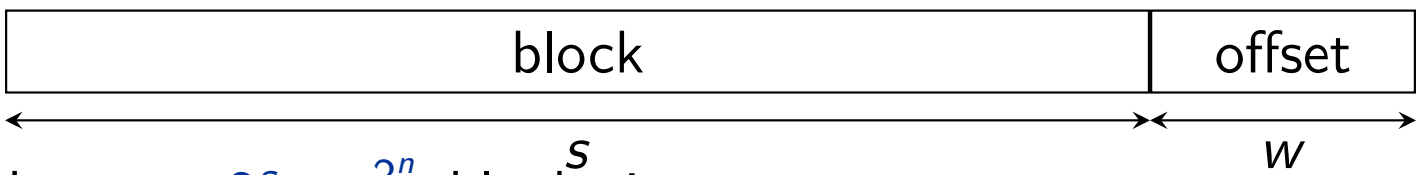
Blocking, Addresses, and Mapping Function

The setup:

- We get read/write requests to addresses.
- Each address is n bits wide.
- Each address specifies an addressable unit (a byte).
- Bytes are grouped into blocks of 2^w bytes per block.
- The cache can only hold m different cache lines.

We can imagine our address is actually consisting of two fields:

- A field of size $s = n - w$ specifying a block number in memory.
- A field of size w specifying an offset inside the block.



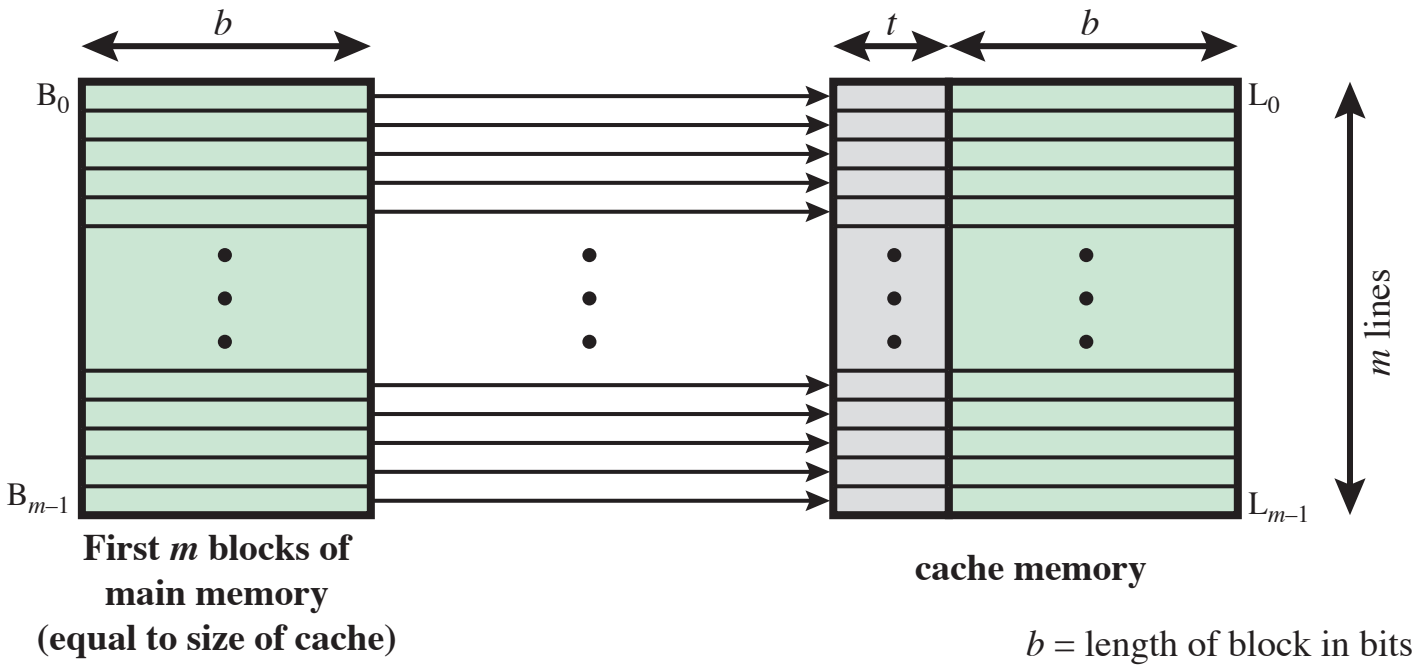
- So there are $2^s = \frac{2^n}{2^w}$ blocks in memory.
- Depending on the mapping function, we will divide the block field further.
- To be consistent with the book we also call the offset field "word".

Mapping Function Overview

- The cache can only hold a relatively small amount of blocks.
- Given a block number in main memory, we must decide in which cache line it can/must go.
- We also need some extra control data for each cache line:
  - "valid bit": do we even have a block stored?
  - "tag": which block have we stored?
  - ...
- Direct mapping
  - The simplest technique.
  - Each block must go to a specific cache line.
- Associative mapping
  - The other extreme: every block can be stored in any cache line.
- Set-associative mapping
  - A compromise: each block must be stored in one of a small set of lines.
  - Direct mapping is a special case: each set of lines has size 1.
  - Associative mapping is a special case: 1 set with all lines.

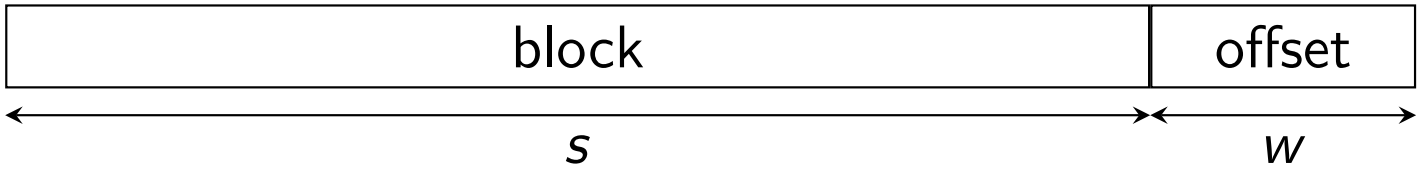
Direct Mapping

- We have $m = 2^r$ cache lines available, numbered 0 to $m - 1$.
- Each of the 2^s memory blocks are assigned a specific line.
- Every time that block is needed it is always placed there.
- Block j is placed in line $i = j \mod m$.
- Block 0, m, $2m$, $3m$, ... are all mapped to line 0.
- Block 1, $1 + m$, $1 + 2m$, $1 + 3m$, ... are all mapped to line 1.
- ...
- So there are $\frac{2^s}{2^r} = 2^{s-r}$ blocks mapped to each line.

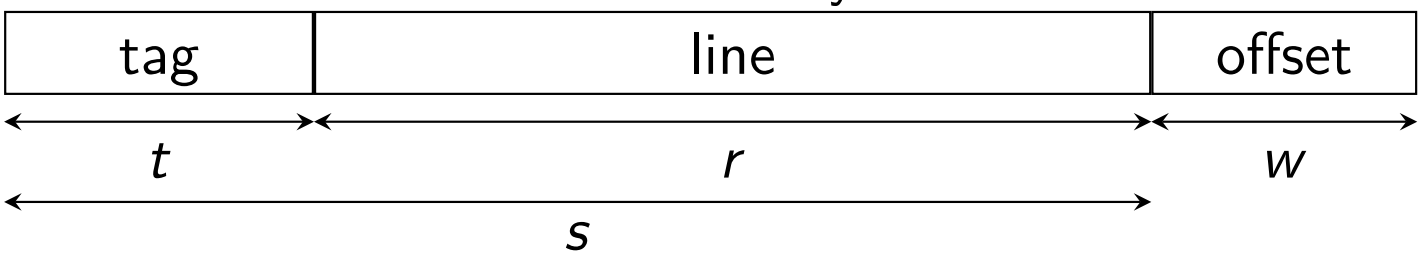


Direct Mapping

- Given an address we can easily determine where it should go.

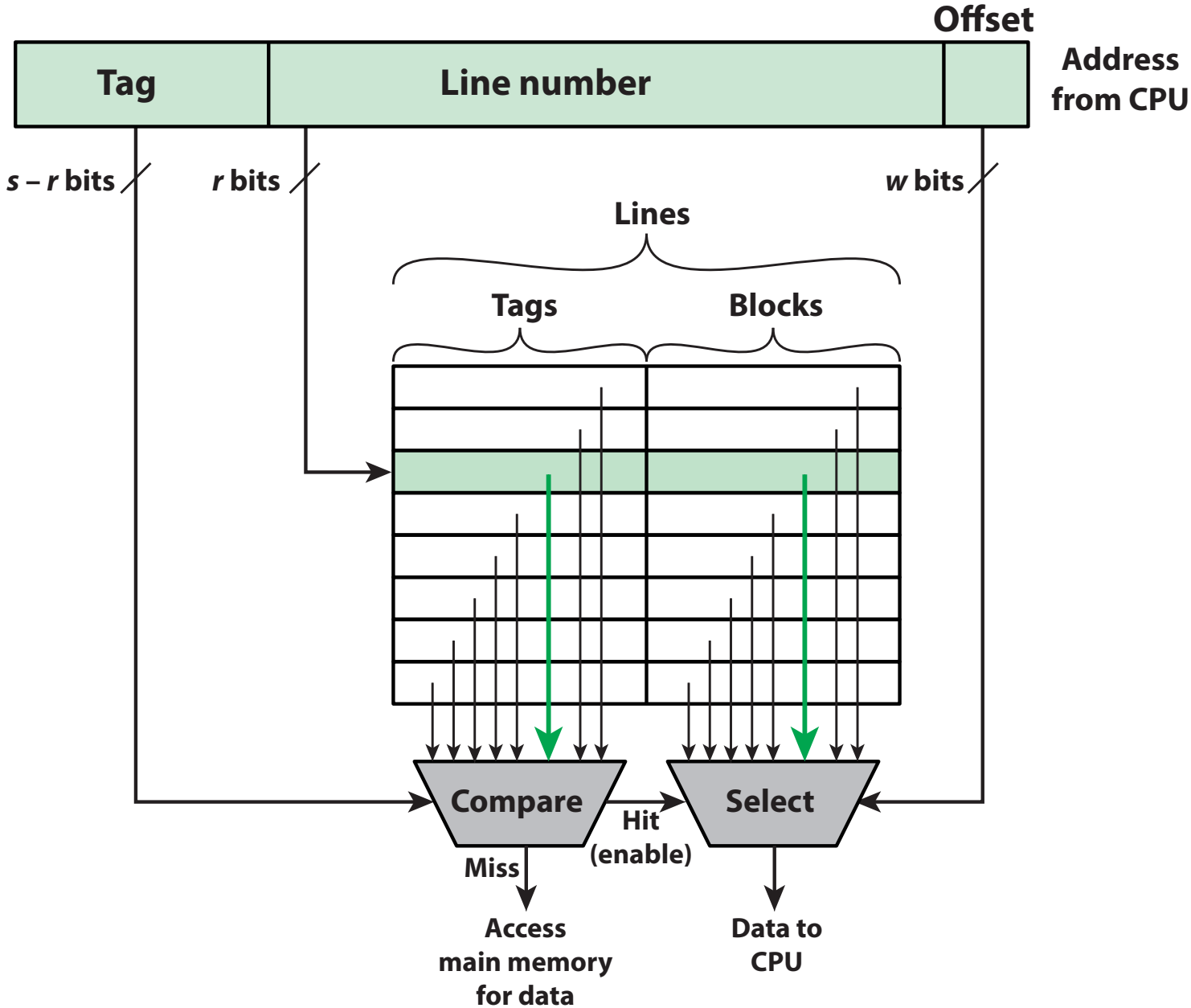


- The block number j is the upper s bits.
- Remember there are $m = 2^r$ cache lines.
- The line number is $i = j \mod m = j \mod 2^r$, which corresponds to the lower r bits of the block number.
- Pretend the block number is actually two fields:

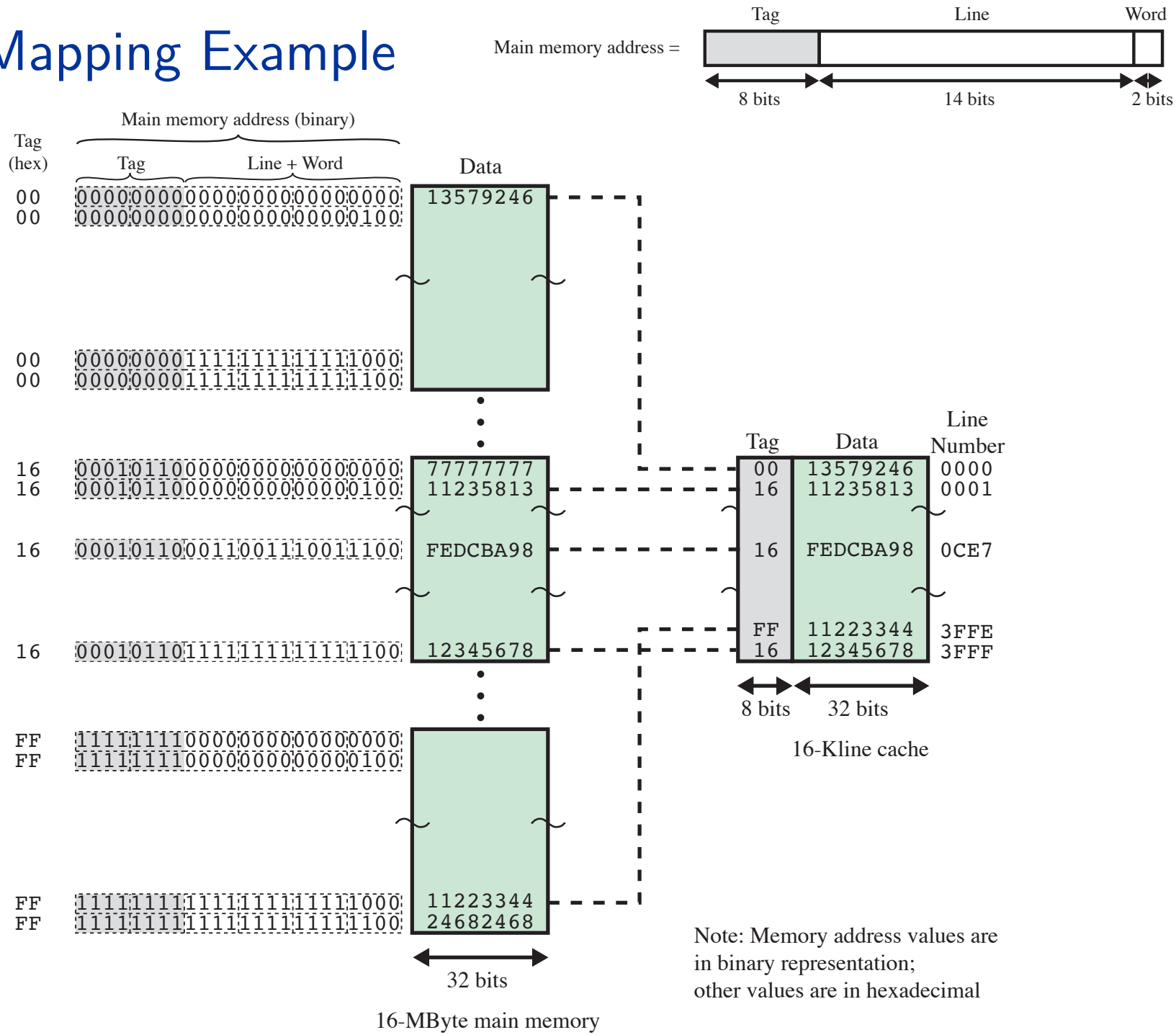


- Which of the $2^{s-r} = 2^t$ blocks are stored in a given line?
- Save the upper t address bits along with the data, the tag.
- Do we have block j in the cache?
- Find the line it should be in, compare the tags.

Direct Mapping Cache Organization

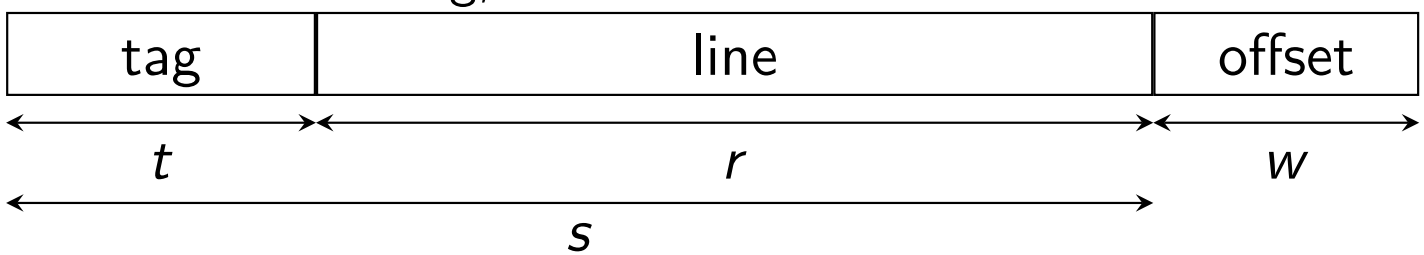


Direct Mapping Example



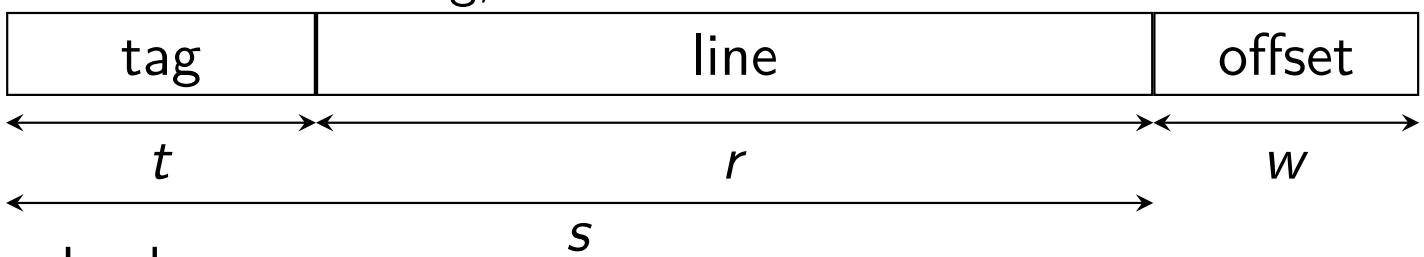
Summary, Direct-Mapped Caches

- Address length: $n = s + w$ bits.
- Lines in cache: $m = 2^r$ lines.
- Block/line size: 2^w bytes (or words).
- Addressable units: 2^n bytes (or words).
- Size of cache: $m \cdot 2^w = 2^{r+w}$ bytes (or words).
- Blocks in main memory: $\frac{2^n}{2^w} = \frac{2^{s+w}}{2^w} = 2^s$ blocks.
- Address format:
  - The lowest w bits encode a byte (or word) in a block.
  - The next r bits encode the cache line to map to.
  - The rest is the tag, to differentiate blocks.



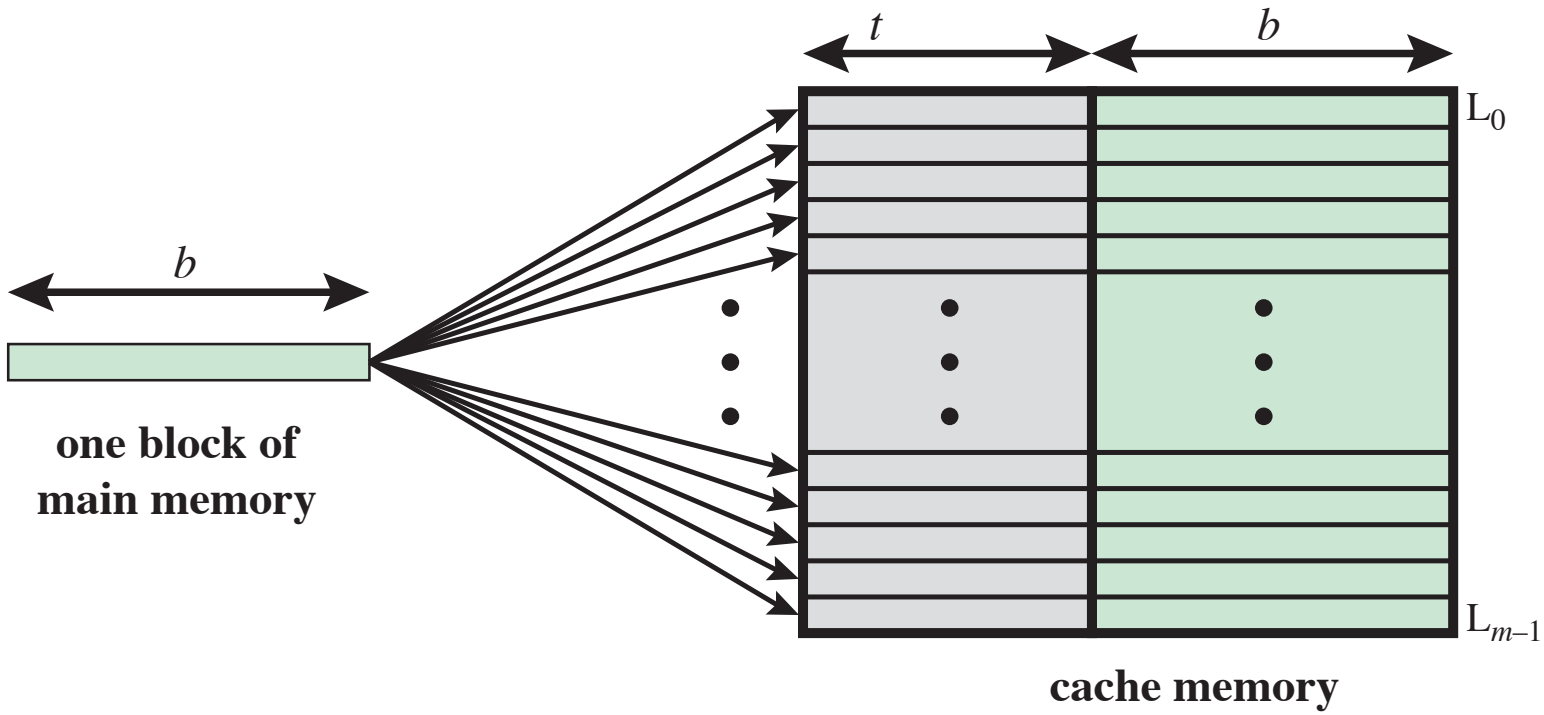
Summary, Direct-Mapped Caches

- Address length: $n = s + w$ bits.
- Lines in cache: $m = 2^r$ lines.
- Block/line size: 2^w bytes (or words).
- Addressable units: 2^n bytes (or words).
- Size of cache: $m \cdot 2^w = 2^{r+w}$ bytes (or words).
- Blocks in main memory: $\frac{2^n}{2^w} = \frac{2^{s+w}}{2^w} = 2^s$ blocks.
- Address format:
  - The lowest w bits encode a byte (or word) in a block.
  - The next r bits encode the cache line to map to.
  - The rest is the tag, to differentiate blocks.

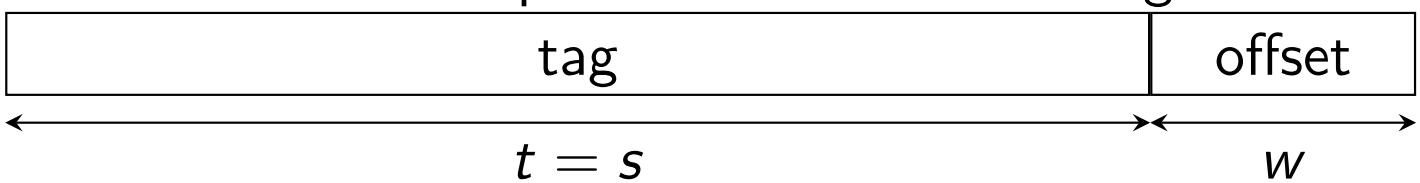


- Drawbacks:
  - A block must be stored in a specific line.
  - If we happen to repeatedly use memory from two blocks mapping to the same line, we swap blocks all the time.
  - This phenomenon is called thrashing.

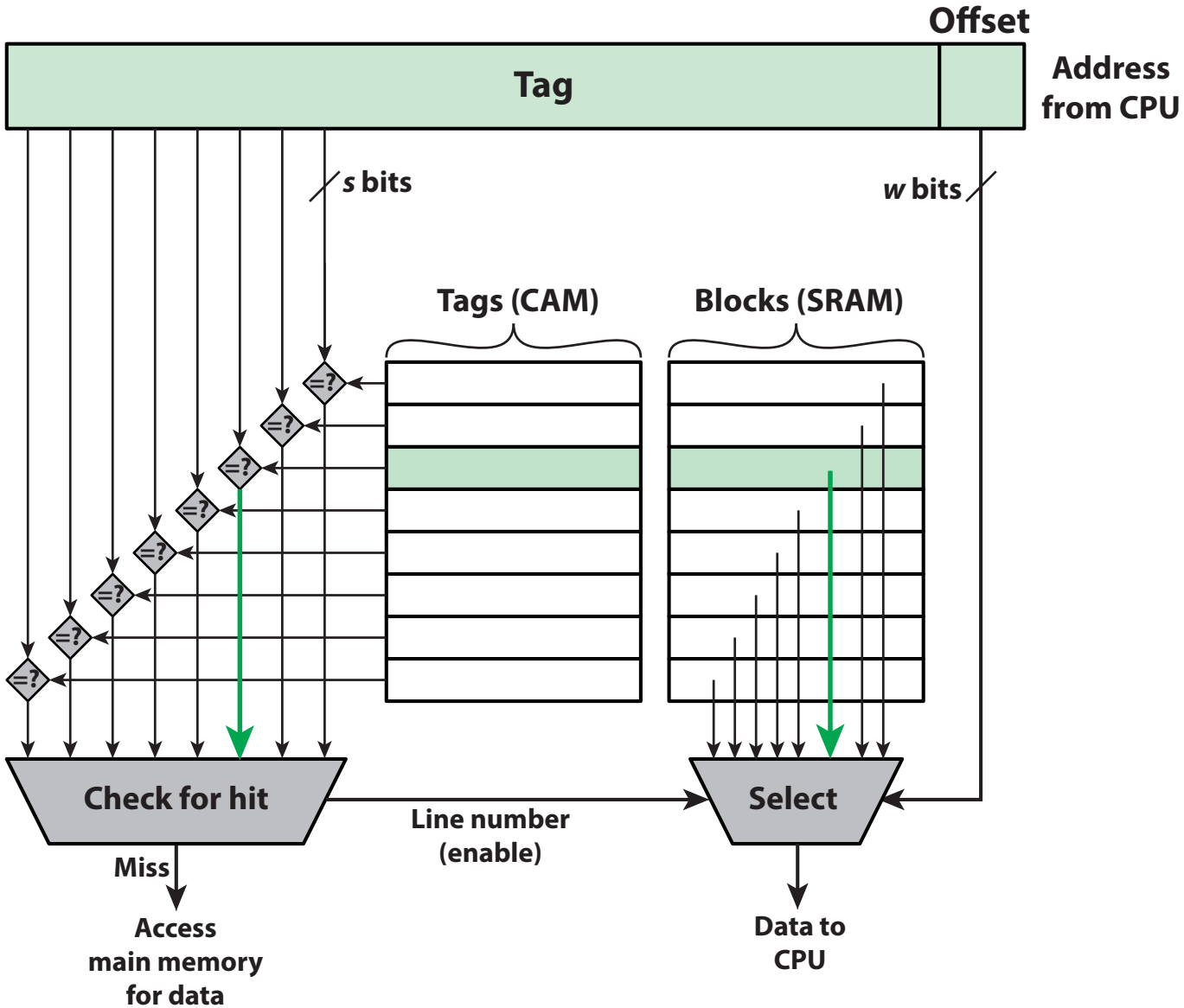
(Fully) Associative Mapping



- A memory block can be put into any cache line.
- We must store the complete block number as tag:

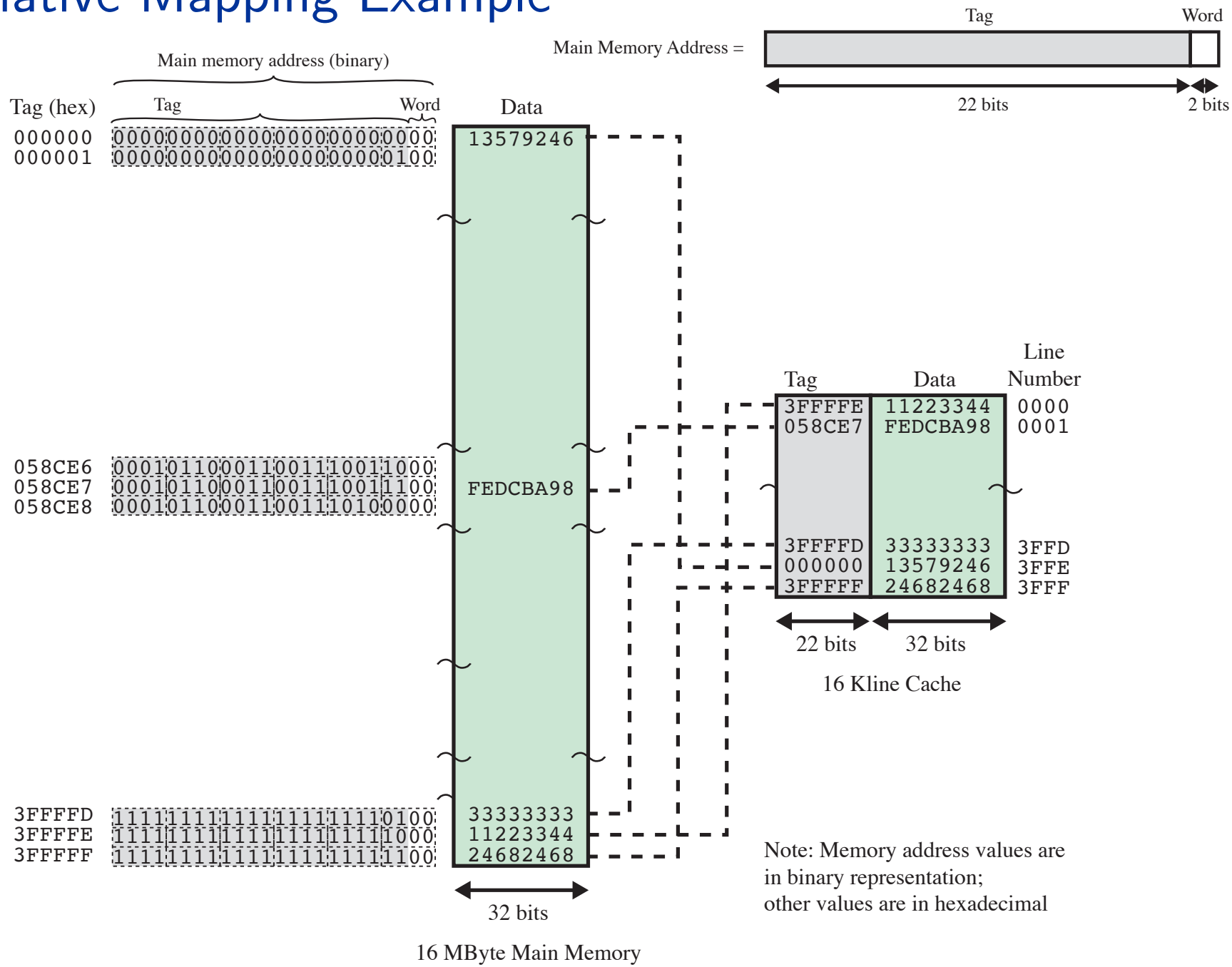


Fully Associative Cache Organization



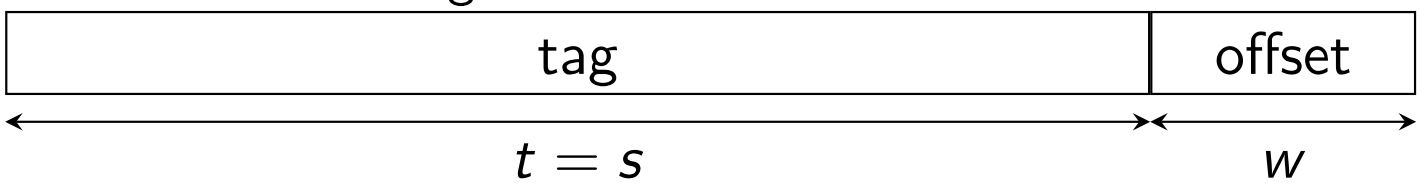
CAM: content-addressable memory

Associative Mapping Example



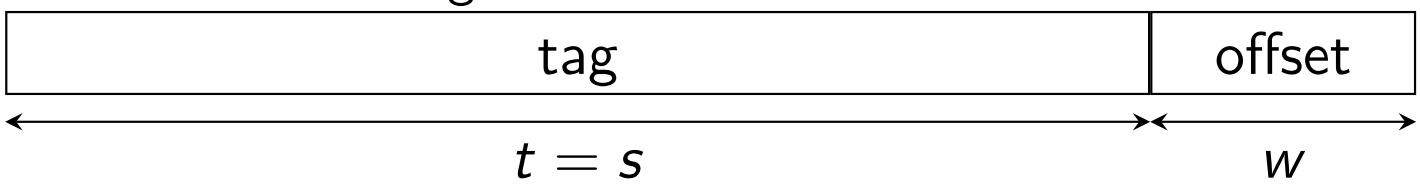
Summary, Associative Caches

- Address length: $n = s + w$ bits.
- Lines in cache: m lines (not necessarily a power of 2).
- Block/line size: 2^w bytes (or words).
- Addressable units: 2^n bytes (or words).
- Size of cache: $m \cdot 2^w$ bytes (or words).
- Blocks in main memory: $\frac{2^n}{2^w} = \frac{2^{s+w}}{2^w} = 2^s$ blocks.
- Address format:
  - The lowest w bits encode a byte/word in a block.
  - The rest is the tag.



Summary, Associative Caches

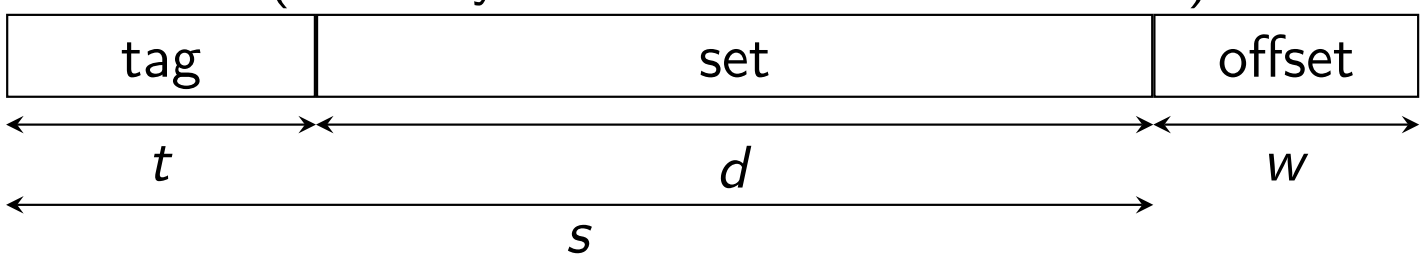
- Address length: $n = s + w$ bits.
- Lines in cache: m lines (not necessarily a power of 2).
- Block/line size: 2^w bytes (or words).
- Addressable units: 2^n bytes (or words).
- Size of cache: $m \cdot 2^w$ bytes (or words).
- Blocks in main memory: $\frac{2^n}{2^w} = \frac{2^{s+w}}{2^w} = 2^s$ blocks.
- Address format:
  - The lowest w bits encode a byte/word in a block.
  - The rest is the tag.



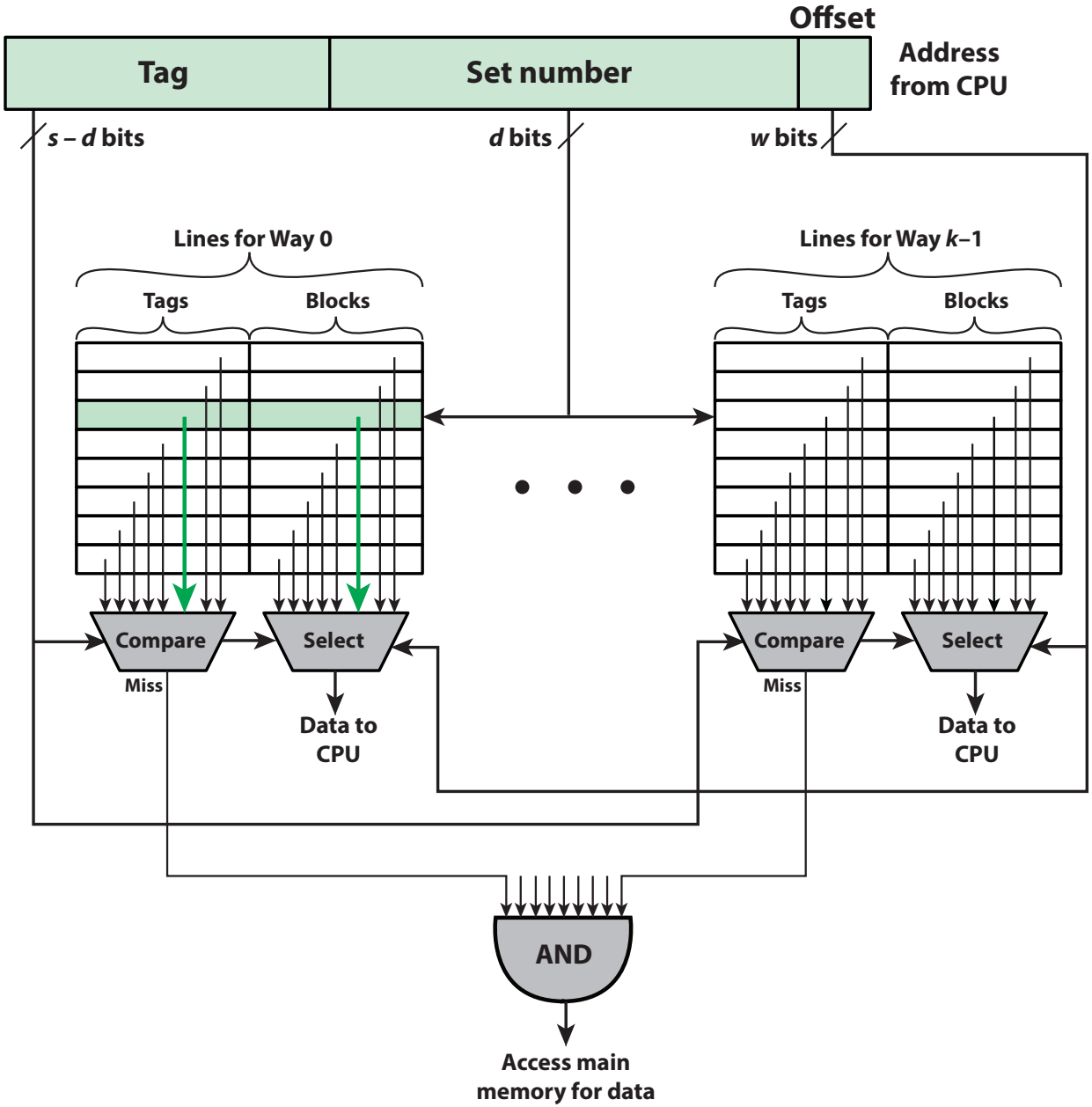
- Notes:
  - When fetching a block into the cache, which line to store it in?
  - Replacement algorithms (see later).
- Drawbacks:
  - The logic for tag comparison becomes very complicated.

Set-Associative Mapping

- Compromise that exhibits the strengths of both the direct and associative approaches, while reducing their disadvantages.
- The m cache lines are now divided into $v = 2^d$ sets with k lines each.
- Instead of numbering the lines, we number the sets.
- Each memory block is mapped to a set.
- E.g., 2 lines per set:
  - "2-way set-associative cache"
  - Each block may be put in either of the 2 lines in a specific set.
- In general:
  - A k-way associative cache has v sets.
  - Total number of lines is $m = k \cdot v$.
  - Block j is mapped to set $i = j \mod v = j \mod 2^d$.
- Address format: (basically the same idea as for direct)

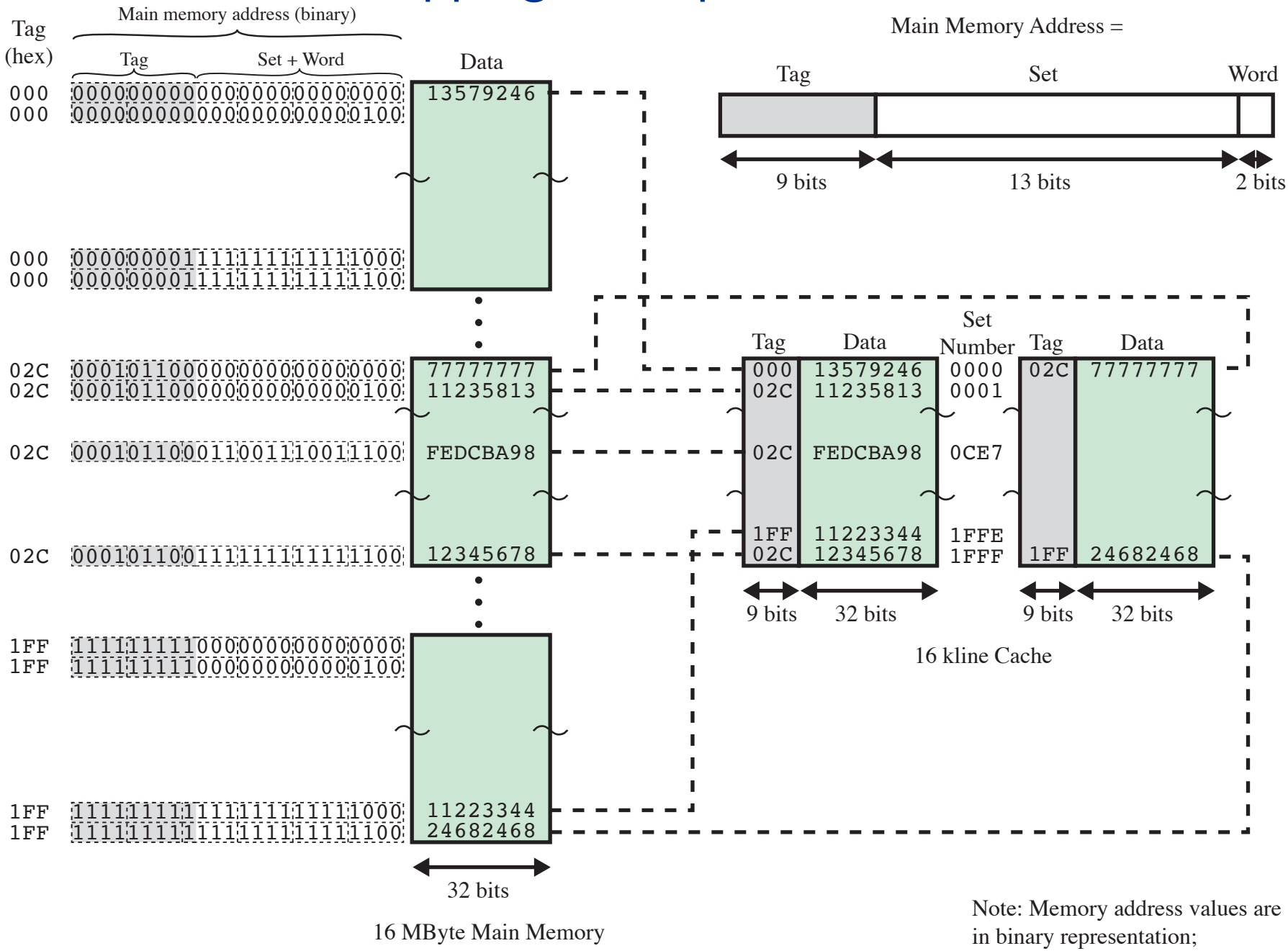


k-Way Set-Associative Cache Organization



Organisation as k direct-mapped caches.

Set-Associative Mapping Example

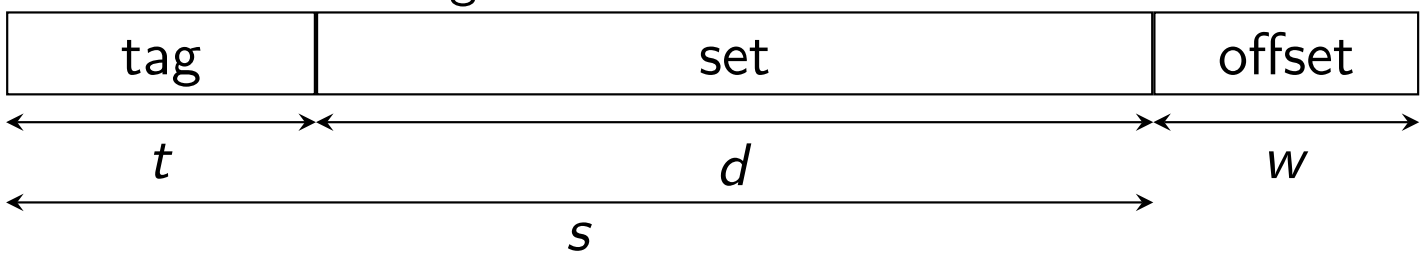


Note: Memory address values are in binary representation; other values are in hexadecimal

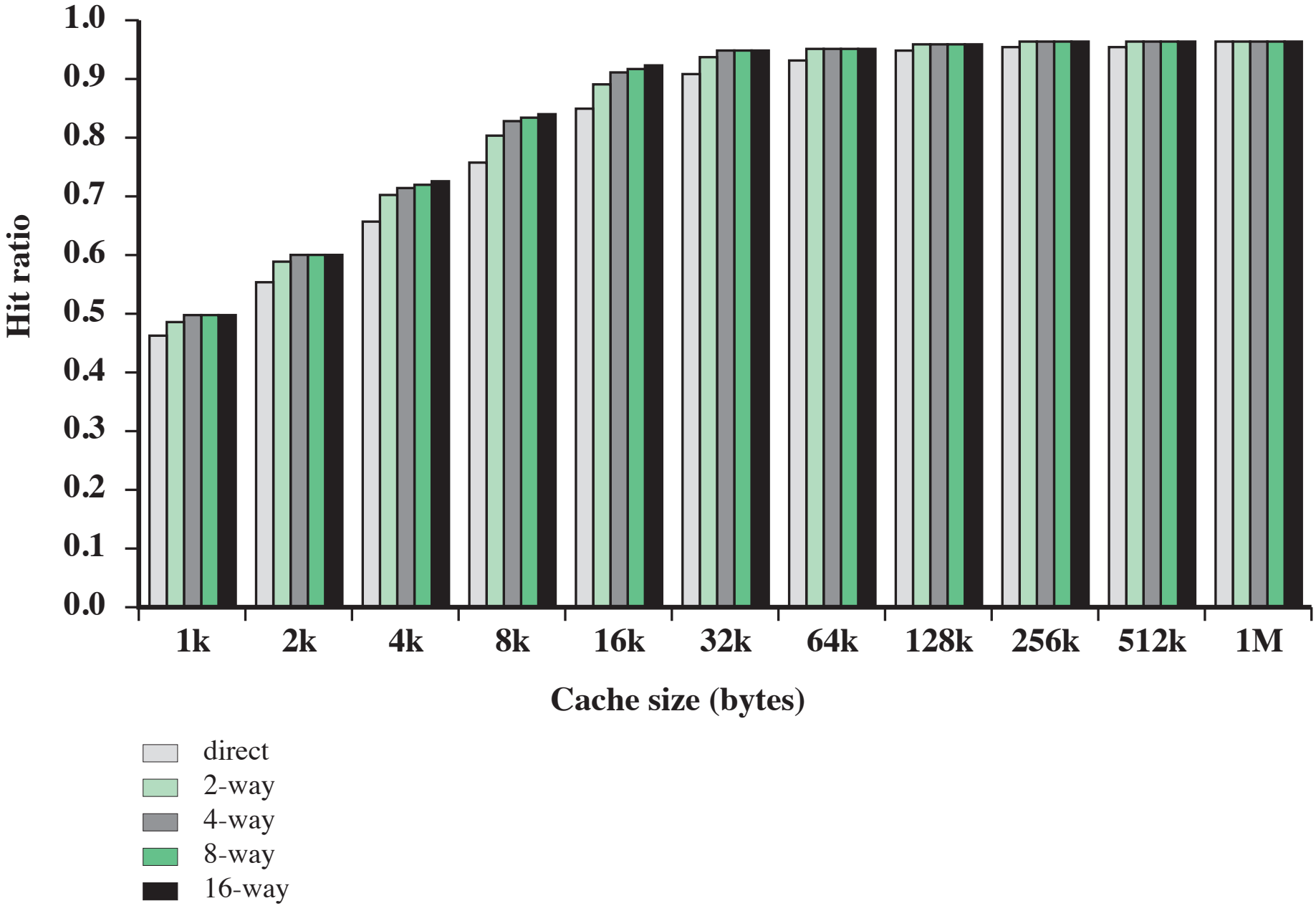
Summary, Set-Associative Caches

- Address length: $n = s + w$ bits.
- Associativity: k lines per set.
- Number of sets: $v = 2^d$ sets.
- Lines in cache: $m = k \cdot v$ lines.
- Block/line size: 2^w bytes (or words).

- Addressable units: 2^n bytes (or words).
- Size of cache: $m \cdot 2^w = k \cdot 2^{d+w}$ bytes (or words).
- Blocks in main memory: $\frac{2^n}{2^w} = \frac{2^{s+w}}{2^w} = 2^s$ blocks.
- Address format:
  - The lowest w bits encode a byte (or word) in a block.
  - The next d bits encode the set in the cache to map to.
  - The rest is the tag.



Performance: Associativity and Cache Size



- The Memory Hierarchy and the Principle of Locality
- Memory Characteristics
- Memory Hierarchy
- Cache Memory Principles
- Cache Design
  - Addresses and Mapping Functions
  - Replacement Algorithms
  - Write Policy
  - Cache Configuration and Hit Ratios

Replacement Algorithms

- What happens when there is no free line to store a block?
- Direct mapping:
  - We only have one choice of line to store it in.
  - If something is there, throw it out.
- Associative and set-associative
  - We have a choice among a set of lines.
  - If non are free, which one to throw out?
- We need a replacement algorithm.
- We need high speed, so it must be implemented in hardware.

It's an online problem:

- We don't know which blocks we need in the future.
- We must make a choice now.
- We can't change our mind later.

There is a whole research area dealing with online problems.
E.g., at IMADA: `https://ola.sdu.dk`

Common Replacement Algorithms

- Least recently used (LRU)
  - Probably most effective.
  - Keep track of when a block has most recently been referenced.
  - Throw the block out which was least recently referenced.
  - Simple to implement, so the most popular.
- First-in-first-out (FIFO)
  - Keep track of when each block was loaded.
  - Throw the block out which was loaded first.
  - Also simple to implement (round-robin or circular buffer).
- Least frequently used (LFU)
  - Keep track of how many times a block has been referenced.
  - Throw the block out with the fewest references.
  - Could be implemented using a counter for each line.
- (Pseudo) Random
  - A bit worse than the others, but still quite good.
  - Expensive to implement.

- The Memory Hierarchy and the Principle of Locality
- Memory Characteristics
- Memory Hierarchy
- Cache Memory Principles
- Cache Design
  - Addresses and Mapping Functions
  - Replacement Algorithms
  - Write Policy
  - Cache Configuration and Hit Ratios

Write Policy

- What should happen when we write data to a location?
- Should we update both the cache and main memory?
- If we only update the cache, we need to be careful when we overwrite the cache line.

The larger picture is even more complicated:

- IO modules may want to read/write to main memory.
- If we don't update main memory, then there may be inconsistencies.
- With multiple CPU cores, each of them may have a cache.
- If we change one cache, the block in the others should be invalidated.

Write Through and Write Back

Write Through:

- Simplest technique.
- All write operations goes both to the cache and main memory.
- Great, main memory is always up to date.
- Other processors listen to writes, and updates/invalidates their caches accordingly.
- However, this generates a substantial amount of memory traffic, a potential bottleneck.

Write Back:

- Minimizes main memory writes.
- Only update the cache, but set a "dirty" bit.
- When a block is replaced, then write updates back to main memory.
- Parts of main memory can now be invalid, so IO modules must access memory through the caches as well.
- The circuitry for this is complex and is a potential bottleneck.

Write Allocation

Another dimension to consider. On a write miss:

- Write Allocate: bring the line into cache and then modify it.
  Commonly used with the Write Back policy, subsequent writes can just modify the cache without a main memory transfer.
- No Write Allocate: send the modification directly to main memory, the block never gets into the cache.
  Commonly used with the Write Through policy, we would anyway write to main memory, so why bring it into the cache?

Some systems, e.g., the ARM Cortex, the allocation policy can be configured.

- The Memory Hierarchy and the Principle of Locality
- Memory Characteristics
- Memory Hierarchy
- Cache Memory Principles
- Cache Design
  - Addresses and Mapping Functions
  - Replacement Algorithms
  - Write Policy
  - Cache Configuration and Hit Ratios

Line Size

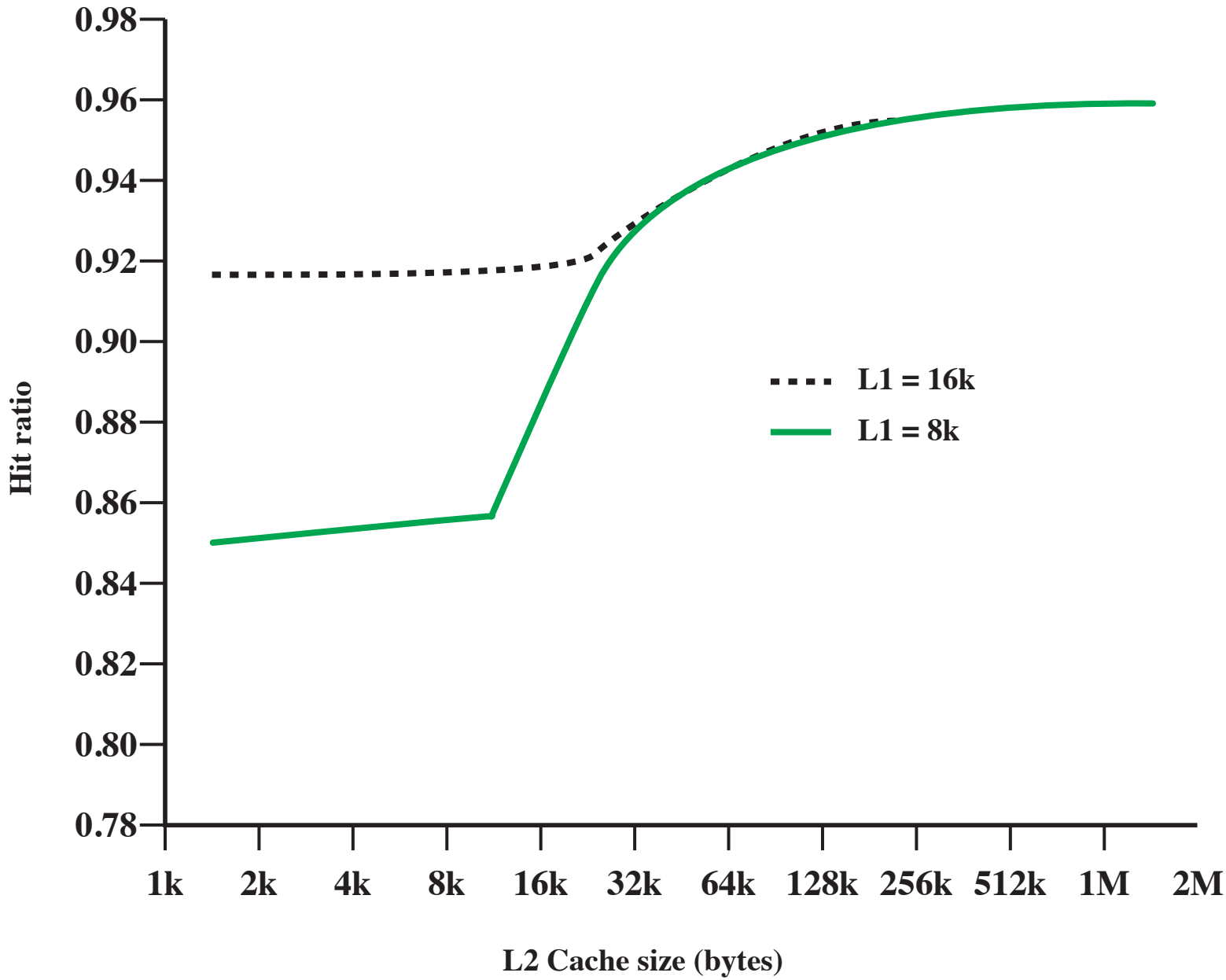
- When we read a byte/word, we get a whole block.
- Chances are we will need many of the neighbouring bytes/words anyway.
- So increasing the block size will initially increase the hit ratio.
- But if increased further the hit ratio will fall again:
  - The chance of using more distant data than the requested is lower.
  - The chance that we needed some data in the block thrown out gets higher.
- Currently a commonly used block size is 64 bytes.

- Programming considerations:
  - Does my object fit within a single cache line?
  - Are multiple threads accessing data within the same cache line?

Multilevel Caches

- In the early days of caches, there was just one, separately from the CPU.
- Now caches can fit in the same chip as the CPU: much faster access.
- We also use multiple caches in a hierarchy:
  - L1: small but fast.
  - L2: larger but further away.
  - ...
  - LLC: "last-level cache"
- It is now common to have L1, L2, L3 on the CPU chip.
- With multiple levels the design issues become more complicated
  - sizes and associativities
  - replacement algorithms
  - write policies
  - allocation policies

Hit Ratio L1 and L2 Cache, Example Simulation

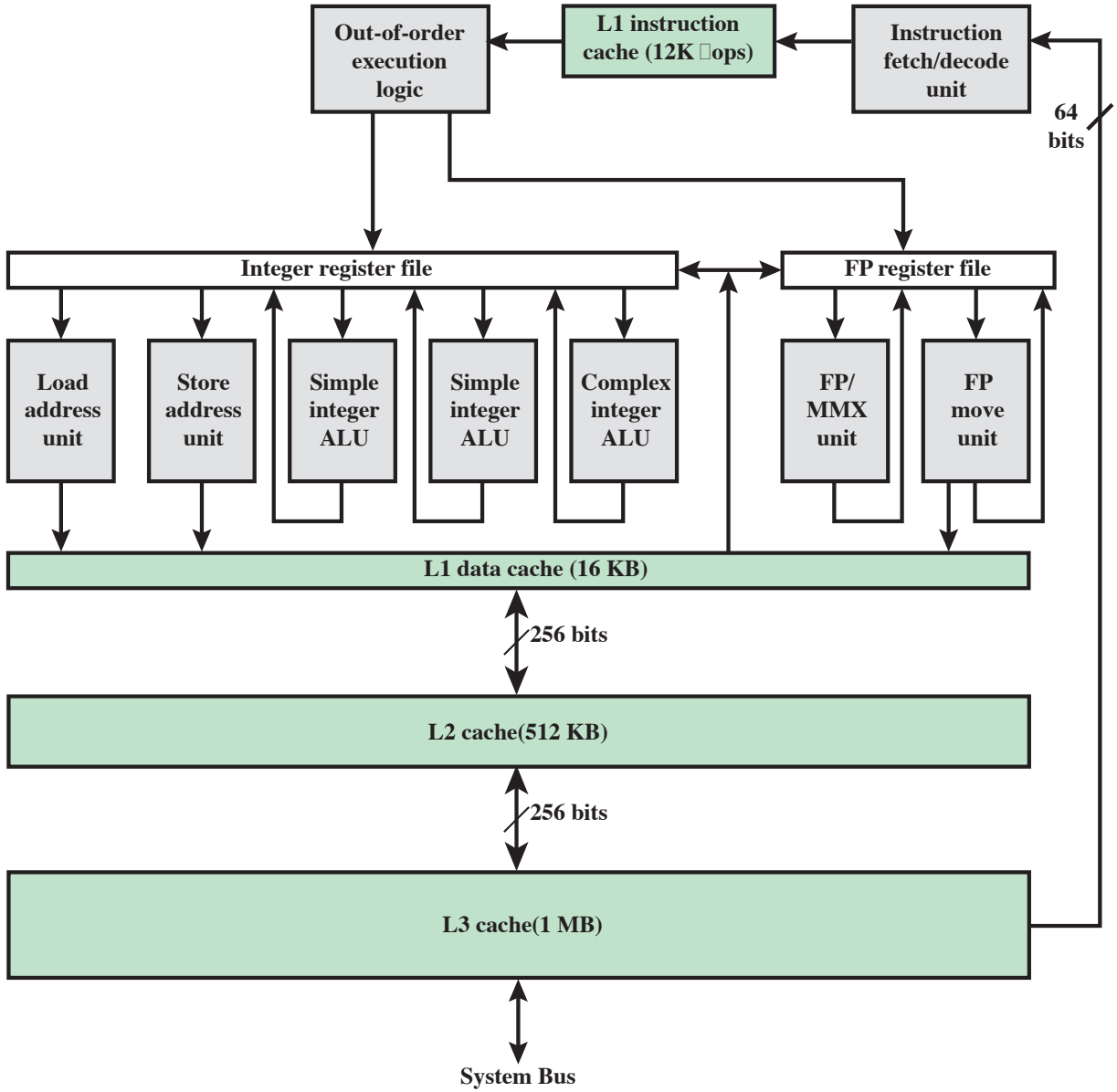


Split Cache or Unified Cache

- The CPU needs two different types: instructions and data.
- They are basically never at the same locations.
- Split Cache
  - Use two caches at the same level.
  - One only to store fetched instructions.
  - One for data read/write.
  - Eliminates cache contention between fetch/decode unit and execution unit.
  - Important for pipelining (much more in later lectures).
- Unified cache
  - Generally higher hit rate.
  - Automatically balances instruction load and data read/write.
  - Only one cache need to be designed and implemented.

Now it is common to have a split L1 cache, but unified cache for higher levels.

Pentium 4 Cache Block Diagram



See `https://en.wikichip.org/` for diagrams of recent CPUs.

Cache Size

Processor	Type	Year	L1 Cache	L2 Cache	L3 Cache
IBM 360/85	MF	1968	16–32 KiB	—	—
PDP-11/70	MC	1975	1 KiB	—	—
IBM 3090	MF	1985	128–256 KiB	—	—
Intel 80486	PC	1989	8 KiB	—	—
Pentium	PC	1993	8/8 KiB	256–512 KiB	—
PowerPC G4	PC/S	1999	32/32 KiB	256–1024 KiB	2 MiB
IBM S/390 G6	MF	1999	256 KiB	8 MiB	—
Pentium 4	PC/S	2000	8/8 KiB	256 KiB	—
Itanium	PC/S	2001	16/16 KiB	96 KiB	4 MiB
Itanium 2	PC/S	2002	32 KiB	256 KiB	6 MiB
IBM POWER5	High-end S	2003	64 KiB	1.9 MiB	36 MiB
IBM POWER6	PC/S	2007	64/64 KiB	4 MiB	32 MiB
IBM z10	MF	2008	64/128 KiB	3 MiB	24–48 MiB
Intel Core i7 EE 990	WS/S	2011	$6 \times 32/32$ KiB	1.5 MiB	12 MiB
IBM zEnterprise 196	MF/S	2011	$24 \times 64/128$ KiB	24×1.5 MiB	24 MiB
					L4: 192 MiB

MF: mainframe
MC: minicomputer
WS: workstation
S: server

Intel Cache Evolution

Problem	Solution	CPU
External memory slower than system bus.	Add external cache using faster memory technology.	386
Increased CPU speed results in external bus becoming a bottleneck for cache access.	Move external cache on chip. Now same speed as the CPU.	486
Internal cache rather small. Limited space on chip.	Add external L2 using faster technology than main memory.	486
Contention occurs, instruction prefetcher and execution unit require access to the cache. The prefetcher is stalled while the execution unit access data.	Create separate data and instruction caches.	Pentium
Increased CPU speed, external bus becomes bottleneck for L2 cache access.	Create separate back-side bus dedicated to L2. Runs at higher speed than main external bus (front-side).	Pentium Pro
	Move L2 cache on chip.	
Some applications deal with massive databases and need rapid access to large amounts of data. The on-chip caches are too small.	Add external L3 cache.	Pentium III
	Move L3 cache on chip.	Pentium 4